ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №5

«Обробка помилок, файли та серіалізація»

з курсу «Професійний Python»
Варіант №3 — «Імпорт та експорт даних інтернет-магазину»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 27.10.2026

Львів — 2026

1. Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із винятками (exceptions), файловими ресурсами, конфігурацією та структурованими форматами даних (CSV, JSON, YAML). Опанувати механізм try/except/else/finally, raise, створення власної ієрархії custom exceptions, exception chaining, принципи EAFP і fail-fast, генераторні context managers через модуль contextlib, роботу з модулями logging та pathlib, потокову обробку даних (streaming I/O), атомарний запис файлів та валідацію вхідних даних.

2. Індивідуальний варіант і постановка задачі

Варіант №3: «Імпорт та експорт даних інтернет-магазину».

Вхідні поля сутності: product_id, name, category, price, quantity.

Вимоги варіанта:

1. Реалізувати потокове читання даних із файлу CSV (streaming CSV import).
2. Забезпечити перевірку коректності ID товару (product_id > 0), непорожньої назви (name), невід'ємної ціни (price >= 0), невід'ємної кількості на складі (quantity >= 0) та належності категорії до списку дозволених.
3. Зчитувати зовнішню конфігурацію у форматі YAML, що містить поріг мінімального залишку (min_stock_threshold), список дозволених категорій (allowed_categories), прапорець режиму обробки (skip_invalid) та налаштування логування.
4. Підтримувати дві політики обробки помилок: Strict mode (переривання при першій помилці валідації) та Tolerant mode (логування попередження, пропуск рядка, продовження роботи).
5. Реалізувати логування невалідних записів і попереджень про низькі залишки товарів (low-stock warning).
6. Реалізувати надійний атомарний експорт відфільтрованих валідних записів у JSON-масив через тимчасовий .tmp файл.
7. Створити власний контекстний менеджер для моніторингу часу виконання та логування результатів роботи.

3. Структура проєкту

Проєкт організовано відповідно до сучасної модульної архітектури:

ecommerce_importer/

├── .gitignore

├── config.yaml

├── data/

│ └── products.csv

├── logs/

```
│   └── application.log
├── output/
│   └── products.json
├── report/
│   └── Вересюк_Ірина_Лаб05_V3.pdf
└── src/
    └── ecommerce_importer/
        ├── __init__.py
        ├── config.py
        ├── exceptions.py
        ├── exporters.py
        ├── logging_config.py
        ├── main.py
        ├── models.py
        ├── readers.py
        ├── services.py
        ├── utils.py
        └── validators.py
```

4. Теоретичні засади та принципи надійності

Ієрархія Custom Exceptions: Створення специфічних для предметної області класів винятків, успадкованих від загального ApplicationError, що дозволяє розділити інфраструктурні збої, помилки конфігурації та проблеми з валідацією окремих записів.

Exception Chaining (raise ... from error): Збереження першопричини помилки (наприклад, низькорівневого ValueError під час парсингу рядка в число) всередині контексту доменного винятку RecordValidationError, що спрощує діагностику трейсбеку.

Fail-fast: Валідація конфігураційного YAML-файлу та наявності вхідних ресурсів на етапі запуску програми, щоб уникнути падіння посеред обробки великого масиву даних.

EAFP (Easier to Ask Forgiveness than Permission): Спроба виконати операцію перетворення типу з перехопленням конкретного типу винятку замість надлишкових попередніх перевірок.

Context Manager (with): Гарантоване закриття файлових дескрипторів навіть у разі аварійного викиду винятків у блоці коду.

Streaming I/O: Використання генераторів Python (yield) для рядкового зчитування CSV, що забезпечує константну пам'ять $O(1)$ незалежно від розміру вхідного файлу.

Атомарний запис (Atomic Write): Запис результатів у проміжний файл з суфіксом .tmp з його наступною заміною через метод replace(), що унеможливлює пошкодження або частковий запис основного файлу при аварійному збої програми.

5. Опис створених модулів та повні лістинги коду

5.1. src/ecommerce_importer/exceptions.py

Модуль реалізує доменну ієрархію винятків проєкту, що бере початок від базового класу ApplicationError. Винятки розділено за призначенням на конфігураційні (ConfigurationError) та помилки обробки даних (DataError, DataImportError, DataExportError). Клас RecordValidationError розширено збереженням контексту дефекту: номера рядка (line_number) та назви некоректного поля (field), а також перевизначено метод __str__ для інформативного форматування повідомлень про помилки валідації.

```
class ApplicationError(Exception):

    """Базовий виняток застосунку."""



class ConfigurationError(ApplicationError):

    """Помилка у файлі конфігурації або її валідації."""



class DataError(ApplicationError):

    """Базовий виняток обробки даних."""



class DataImportError(DataError):
```

```
    """Помилка відкриття або читання вхідного джерела."""


class DataExportError(DataError):
    """Помилка запису даних під час експорту."""


class RecordValidationError(DataError):
    """Помилка валідації окремого запису товару."""

    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field

    def __str__(self) -> str:
        details = []
        if self.line_number is not None:
            details.append(f"line={self.line_number}")
        if self.field is not None:
            details.append(f"field='{self.field}'")
```

```
        if details:
            return f"{super().__str__()} ({', '.join(details)})"
        return super().__str__()
```

5.2. src/ecommerce_importer/models.py

Модуль описує строгі моделі даних предметної області та типізовані структури конфігурації. Усі сутності створено за допомогою декоратора @dataclass(frozen=True, slots=True), що гарантує незмінність об'єктів (immutability), захист від випадкових мутацій стану та оптимізацію використання пам'яті. Клас Product інкапсулює атрибути товару та надає метод to_dict() для подальшої серіалізації, а датакласи ProcessingConfig, LoggingConfig та AppConfig формують типізовану ієрархію налаштувань застосунку.

```
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any


@dataclass(frozen=True, slots=True)
class Product:
    """Доменний об'єкт товару інтернет-магазину."""
    product_id: int
    name: str
    category: str
    price: float
    quantity: int

    def to_dict(self) -> dict[str, Any]:
        return asdict(self)


@dataclass(frozen=True, slots=True)
```

```
class ProcessingConfig:
    min_stock_threshold: int
    allowed_categories: set[str]
    skip_invalid: bool


@dataclass(frozen=True, slots=True)
class LoggingConfig:
    level: str
    path: Path


@dataclass(frozen=True, slots=True)
class AppConfig:
    input_path: Path
    output_path: Path
    processing: ProcessingConfig
    logging: LoggingConfig
```

5.3. src/ecommerce_importer/config.py

Модуль забезпечує зчитування та валідацію зовнішньої конфігурації у форматі YAML із дотриманням принципу fail-fast. Функція load_config використовує безпечний парсер yaml.safe_load() для запобігання виконанню довільного коду та здійснює мапінг сирих словникових даних у типізований об'єкт AppConfig. Функція validate_config перевіряє наявність вхідного CSV-файлу через pathlib.Path.exists() та валідність числових меж параметрів ще до початку читання даних, сигналізуючи про відхилення винятком ConfigurationError.

```
from pathlib import Path
import yaml

from ecommerce_importer.exceptions import ConfigurationError
```

```
from ecommerce_importer.models import AppConfig, LoggingConfig, ProcessingConfig


def validate_config(config: AppConfig) -> None:
    """Перевірка коректності конфігураційних значень."""
    if config.processing.min_stock_threshold < 0:
        raise ConfigurationError("min_stock_threshold cannot be negative")
    if not config.processing.allowed_categories:
        raise ConfigurationError("allowed_categories list cannot be empty")
    if not config.input_path.exists():
        raise ConfigurationError(f"Input file not found: {config.input_path}")


def load_config(path: Path) -> AppConfig:
    """Безпечне читання та валідація структури YAML конфігурації."""
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Config file not found: {path}") from error
    except yaml.YAMLError as error:
        raise ConfigurationError(f"YAML parsing error in {path}") from error

    if not isinstance(raw, dict):
        raise ConfigurationError("Root YAML element must be a dictionary")

    try:
```

```
    raw_proc = raw["processing"]
    allowed_cats = {str(cat).strip().lower() for cat in raw_proc["allowed_categories"]}

    config = AppConfig(
      input_path=Path(raw["input"]["path"]),
      output_path=Path(raw["output"]["path"]),
      processing=ProcessingConfig(
        min_stock_threshold=int(raw_proc["min_stock_threshold"]),
        allowed_categories=allowed_cats,
        skip_invalid=bool(raw_proc["skip_invalid"]),
      ),
      logging=LoggingConfig(
        level=str(raw["logging"]["level"]),
        path=Path(raw["logging"]["path"]),
      ),
    )
  except (KeyError, TypeError, ValueError) as error:
    raise ConfigurationError("Configuration schema is invalid or has missing keys") from error

  validate_config(config)
  return config
```

5.4. src/ecommerce_importer/logging_config.py

Модуль відповідає за централізовану ініціалізацію підсистеми журналювання logging. Функція configure_logging автоматично створює необхідне дерево каталогів під файл логів за допомогою Path.mkdir(parents=True, exist_ok=True) і налаштовує форматований запис подій із мітками точного часу, рівнем важливості та іменем

модуля-джерела. Логи одночасно транслюються у консоль (StreamHandler) та зберігаються у текстовий файл із фіксованим кодуванням UTF-8 (FileHandler).

```
import logging

from pathlib import Path


def configure_logging(level: str, path: Path) -> None:

  """Налаштовує логування у файл та консоль."""

  path.parent.mkdir(parents=True, exist_ok=True)

  numeric_level = getattr(logging, level.upper(), logging.INFO)


  logging.basicConfig(

    level=numeric_level,

    format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",

    handlers=[

      logging.FileHandler(path, encoding="utf-8"),

      logging.StreamHandler(),

    ],

    force=True,

  )
```

5.5. src/ecommerce_importer/utils.py

Модуль містить власний контекстний менеджер logged_operation, реалізований за допомогою декоратора @contextmanager із бібліотеки contextlib. Менеджер огортає критичні операції конвеєра: фіксує початок операції в лозі на рівні INFO, вимірює тривалість її виконання за допомогою лічильника perf_counter(), у разі штатного завершення (else) виводить затрачений час, а в разі збою (except) фіксує аварійний трейсбек через logger.exception() і повторно піднімає виняток вище.

```
from collections.abc import Iterator

from contextlib import contextmanager
```

```
import logging

from time import perf_counter


logger = logging.getLogger(__name__)




@contextmanager

def logged_operation(operation: str) -> Iterator[None]:

  """Контекстний менеджер для таймінгу та діагностики виконання операції."""

  logger.info("%s started", operation)

  start = perf_counter()

  try:

    yield

  except Exception:

    logger.exception("%s failed due to an error", operation)

    raise

  else:

    elapsed = perf_counter() - start

    logger.info("%s finished successfully in %.6f s", operation, elapsed)
```

5.6. src/ecommerce_importer/readers.py

Модуль реалізує потокове (streaming) зчитування даних із CSV-джерела за допомогою csv.DictReader без завантаження всього файлу в оперативну пам'ять. Функція read_csv_rows оформлена як генератор: вона відкриває файл через контекстний менеджер із зазначенням кодування UTF-8, нумерує рядки починаючи з другого (враховуючи заголовок таблиці) та порційно передає пари (line_number, row) через yield. Усі низькорівневі помилки доступу до файлової системи (OSError) загортаються в доменний виняток DataImportError.

```
from collections.abc import Iterator
```

```
import csv

from pathlib import Path


from ecommerce_importer.exceptions import DataImportError


def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
  """Потокове читання рядків файлу CSV з генерацією (line_number, row)."""
  try:
    with path.open("r", encoding="utf-8", newline="") as file:
      reader = csv.DictReader(file)
      for line_number, row in enumerate(reader, start=2):
        yield line_number, row
  except OSError as error:
    raise DataImportError(f"Cannot read input file: {path}") from error
```

5.7. src/ecommerce_importer/validators.py

Модуль інкапсулює правила бізнес-валідації окремих записів згідно з вимогами Варіанта №3. Функція parse_product перевіряє наявність обов'язкових полів, додатність product_id, непорожність назви товару, належність категорії до множини дозволених (allowed_categories), а також невід'ємність числових параметрів ціни (price >= 0) та залишку на складі (quantity >= 0). При помилках перетворення типів застосовується механізм exception chaining (from error), що зв'язує низькорівневий ValueError із доменним RecordValidationError.

```
from ecommerce_importer.exceptions import RecordValidationError

from ecommerce_importer.models import Product


def parse_product(
  row: dict[str, str],
  *,
```

```
    line_number: int,
    allowed_categories: set[str],
) -> Product:
    """Парсинг та валідація полів сутності Product згідно з бізнес-правилами."""
    # product_id
    try:
        product_id = int(row.get("product_id", "").strip())
        if product_id <= 0:
            raise ValueError("product_id must be a positive integer")
    except (TypeError, ValueError) as error:
        raise RecordValidationError(
            "Invalid product ID",
            line_number=line_number,
            field="product_id",
        ) from error

    # name
    name = row.get("name", "").strip()
    if not name:
        raise RecordValidationError(
            "Product name cannot be empty",
            line_number=line_number,
            field="name",
        )
```

```
# category
category = row.get("category", "").strip().lower()
if not category:
    raise RecordValidationError(
        "Category cannot be empty",
        line_number=line_number,
        field="category",
    )
if category not in allowed_categories:
    raise RecordValidationError(
        f"Category '{category}' is not allowed (permitted: {sorted(allowed_categories)})",
        line_number=line_number,
        field="category",
    )

# price >= 0
try:
    price = float(row.get("price", "").strip())
except (TypeError, ValueError) as error:
    raise RecordValidationError(
        "Invalid price format",
        line_number=line_number,
        field="price",
    ) from error
```

```
if price < 0:
    raise RecordValidationError(
        "Price must be non-negative (>= 0)",
        line_number=line_number,
        field="price",
    )

# quantity >= 0
try:
    quantity = int(row.get("quantity", "").strip())
except (TypeError, ValueError) as error:
    raise RecordValidationError(
        "Invalid quantity format",
        line_number=line_number,
        field="quantity",
    ) from error

if quantity < 0:
    raise RecordValidationError(
        "Quantity must be non-negative (>= 0)",
        line_number=line_number,
        field="quantity",
    )

return Product(
```

```
        product_id=product_id,
        name=name,
        category=category,
        price=price,
        quantity=quantity,
    )
```

5.8. src/ecommerce_importer/exporters.py

Модуль реалізує потоковий та атомарний експорт колекції об'єктів Product у результуючий JSON-файл. Для унеможливлення пошкодження цільового файлу під час раптового збою дані спочатку записуються у проміжний тимчасовий файл із розширенням .tmp. Після завершення формування валідного JSON-масиву виконується атомарна заміна файлів методом temporary.replace(path). У разі виникнення помилки запису тимчасовий файл гарантовано видаляється (temporary.unlink(missing_ok=True)), а помилка прокидається як DataExportError.

```
from collections.abc import Iterable
import json
from pathlib import Path

from ecommerce_importer.exceptions import DataExportError
from ecommerce_importer.models import Product

def export_products_json(products: Iterable[Product], path: Path) -> int:
    """Атомарний потоковий експорт об'єктів Product у JSON-масив."""
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    count = 0

    try:
```

```
    with temporary.open("w", encoding="utf-8") as file:
      file.write("[\n")
      first = True
      for product in products:
        if not first:
          file.write(",\n")
        json.dump(product.to_dict(), file, ensure_ascii=False, indent=2)
        first = False
        count += 1
      file.write("\n]\n")

    # Атомарна підміна цільового файлу
    temporary.replace(path)
  except (OSError, TypeError) as error:
    temporary.unlink(missing_ok=True)
    raise DataExportError(f"Failed to export data to {path}") from error

  return count
```

5.9. src/ecommerce_importer/services.py

Модуль містить сервісний шар застосунку, структуру для збору агрегованої статистики ImportStatistics та генераторну функцію import_products. Модуль оркеструє конвеєр обробки: веде підрахунок загальної кількості рядків, валідних і невалідних записів, перевіряє поріг мінімального залишку товару (min_stock_threshold) із фіксацією попереджень low-stock у лог. Також модуль підтримує перемикання між суворим режимом (Strict mode, де помилка негайно зупиняє роботу) та толерантним (Tolerant mode, де невалідний рядок логується і пропускається).

```
from collections.abc import Iterator

from dataclasses import dataclass
```

```
import logging
from pathlib import Path

from ecommerce_importer.exceptions import RecordValidationError
from ecommerce_importer.models import Product
from ecommerce_importer.readers import read_csv_rows
from ecommerce_importer.validators import parse_product

logger = logging.getLogger(__name__)

@dataclass
class ImportStatistics:
    total: int = 0
    valid: int = 0
    invalid: int = 0
    low_stock_count: int = 0

def import_products(
    path: Path,
    *,
    allowed_categories: set[str],
    min_stock_threshold: int,
    skip_invalid: bool,
    statistics: ImportStatistics,
) -> Iterator[Product]:
```

```
  """Потоковий імпорт та перевірка товарів із відстеженням low-stock та невалідних записів."""

  for line_number, row in read_csv_rows(path):

    statistics.total += 1

    try:

      product = parse_product(

        row,

        line_number=line_number,

        allowed_categories=allowed_categories,

      )

    except RecordValidationError as error:

      statistics.invalid += 1

      logger.warning("Invalid record at line %s: %s", error.line_number, error)

      if skip_invalid:

        continue

      raise  # Strict mode: негайне переривання


    # Перевірка порогу низьких залишків

    if product.quantity <= min_stock_threshold:

      statistics.low_stock_count += 1

      logger.warning(

        "Low stock warning: Product ID=%s ('%s') has only %s items left (threshold=%s)",

        product.product_id,

        product.name,

        product.quantity,

        min_stock_threshold,
```

```
        )


    statistics.valid += 1

    yield product
```

5.10. src/ecommerce_importer/main.py

Головний виконавчий модуль застосунку, що виступає єдиною точкою входу (entry point). Модуль завантажує та валідує YAML-конфігурацію, налаштовує систему логування, ініціалізує лічильники статистики та запускає повний конвеєр потокового імпорту, валідації й атомарного експорту товарів у межах власного контекстного менеджера logged_operation. Усі високорівневі доменні помилки (ApplicationError) централізовано перехоплюються з виведенням повідомлення у журнал та коректним завершенням програми з кодом повернення 1.

```
import logging

from pathlib import Path

import sys


# Додавання шляху до модулів у sys.path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))


from ecommerce_importer.config import load_config

from ecommerce_importer.exceptions import ApplicationError

from ecommerce_importer.exporters import export_products_json

from ecommerce_importer.logging_config import configure_logging

from ecommerce_importer.services import ImportStatistics, import_products

from ecommerce_importer.utils import logged_operation


logger = logging.getLogger("ecommerce_importer")
```

```
def main() -> None:
    try:
        config = load_config(Path("config.yaml"))
        configure_logging(config.logging.level, config.logging.path)
        statistics = ImportStatistics()

        with logged_operation("E-commerce inventory pipeline"):
            products = import_products(
                config.input_path,
                allowed_categories=config.processing.allowed_categories,
                min_stock_threshold=config.processing.min_stock_threshold,
                skip_invalid=config.processing.skip_invalid,
                statistics=statistics,
            )
            exported_count = export_products_json(
                products,
                config.output_path,
            )

        logger.info(
            "Execution completed: Total=%s, valid=%s, invalid=%s, low_stock=%s,
exported=%s",
            statistics.total,
            statistics.valid,
            statistics.invalid,
            statistics.low_stock_count,
```

```
            exported_count,
        )
    except ApplicationError as error:
        logger.error("Application error: %s", error)
        raise SystemExit(1) from error


if __name__ == "__main__":
    main()
```

6. Вміст конфігураційних та вхідних файлів

6.1. config.yaml

```
input:
  path: data/products.csv
output:
  path: output/products.json
processing:
  min_stock_threshold: 5
  allowed_categories:
    - electronics
    - books
    - clothing
    - home
  skip_invalid: true
logging:
  level: INFO
  path: logs/application.log
```

6.2. data/products.csv

product_id,name,category,price,quantity

101,Wireless Mouse,electronics,25.99,15

102,Python Cookbook,books,45.00,3

103,,electronics,99.99,10

104,Desk Lamp,home,30.50,-2

abc,Office Chair,furniture,120.00,5

106,Winter Jacket,clothing,89.90,2

107,Coffee Maker,kitchen,55.00,8

108,Mechanical Keyboard,electronics,-15.00,20

109,T-Shirt,clothing,19.99,50

7. Робота з Git

У репозиторії виконано три послідовні змістовні коміти:

Коміт 1: Базова структура, винятки та моделі даних (models.py, exceptions.py).

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git add .gitignore config.yaml src/ecommerce_importer/__init__.py src/ecommerce_importer/exceptions.py src/ecommerce_importer/models.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git commit -m "Initialize project structure, custom exceptions and domain models"

[main (root-commit) 55b227e] Initialize project structure, custom exceptions and domain models

5 files changed, 106 insertions(+)

create mode 100644 .gitignore

create mode 100644 config.yaml

create mode 100644 src/ecommerce_importer/__init__.py

create mode 100644 src/ecommerce_importer/exceptions.py

create mode 100644 src/ecommerce_importer/models.py

Коміт 2: Інфраструктура логування, контекстний менеджер, потокове читання та валідація.

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git add src/ecommerce_importer/config.py src/ecommerce_importer/logging_config.py src/ecommerce_importer/utils.py src/ecommerce_importer/readers.py src/ecommerce_importer/validators.py data/products.csv

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git commit -m "Implement streaming reader, strict/tolerant validators and logging utilities"

[main 8cd2673] Implement streaming reader, strict/tolerant validators and logging utilities

6 files changed, 205 insertions(+)

create mode 100644 data/products.csv

create mode 100644 src/ecommerce_importer/config.py

create mode 100644 src/ecommerce_importer/logging_config.py

create mode 100644 src/ecommerce_importer/readers.py

create mode 100644 src/ecommerce_importer/utils.py

create mode 100644 src/ecommerce_importer/validators.py

Коміт 3: Атомарний експорт, сервісний шар та головна точка входу.

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git add src/ecommerce_importer/exporters.py src/ecommerce_importer/services.py src/ecommerce_importer/main.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git commit -m "Add atomic JSON exporter, pipeline services and application entry point"

[main 1e40398] Add atomic JSON exporter, pipeline services and application entry point

3 files changed, 142 insertions(+)

create mode 100644 src/ecommerce_importer/exporters.py

create mode 100644 src/ecommerce_importer/main.py

create mode 100644 src/ecommerce_importer/services.py

7.2. Результат виконання git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git status

On branch main

nothing to commit, working tree clean

7.3. Результат виконання git log –oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/ecommerce_importer (main)

$ git log --oneline

1e40398 (**HEAD** -> **main**) Add atomic JSON exporter, pipeline services and application entry point

8cd2673 Implement streaming reader, strict/tolerant validators and logging utilities

55b227e Initialize project structure, custom exceptions and domain models

8. Результати запуску програми

8.1. Фактичне виведення у консолі (Tolerant mode)

PS D:\Робочий стіл\проф. python\ecommerce_importer> Get-Content logs/application.log

2026-10-27 20:02:54,722 | INFO | ecommerce_importer.utils | E-commerce inventory pipeline started

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Low stock warning: Product ID=102 ('Python Cookbook') has only 3 items left (threshold=5)

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Invalid record at line 4: Product name cannot be empty (line=4, field='name')

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Invalid record at line 5: Quantity must be non-negative (>= 0) (line=5, field='quantity')

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Invalid record at line 6: Invalid product ID (line=6, field='product_id')

2026-10-27 20:02:54,724 | WARNING | ecommerce_importer.services | Low stock warning: Product ID=106 ('Winter Jacket') has only 2 items left (threshold=5)

2026-10-27 20:02:54,724 | WARNING | ecommerce_importer.services | Invalid record at line 8: Category 'kitchen' is not allowed (permitted: ['books', 'clothing', 'electronics', 'home']) (line=8, field='category')

2026-10-27 20:02:54,724 | WARNING | ecommerce_importer.services | Invalid record at line 9: Price must be non-negative (>= 0) (line=9, field='price')

2026-10-27 20:02:54,728 | INFO | ecommerce_importer.utils | E-commerce inventory pipeline finished successfully in 0.005455 s

2026-10-27 20:02:54,728 | INFO | ecommerce_importer | Execution completed: Total=9, valid=4, invalid=5, low_stock=2, exported=4

8.2. Вміст файлу логів logs/application.log

2026-10-27 20:02:54,722 | INFO | ecommerce_importer.utils | E-commerce inventory pipeline started

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Low stock warning: Product ID=102 ('Python Cookbook') has only 3 items left (threshold=5)

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Invalid record at line 4: Product name cannot be empty (line=4, field='name')

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Invalid record at line 5: Quantity must be non-negative (>= 0) (line=5, field='quantity')

2026-10-27 20:02:54,723 | WARNING | ecommerce_importer.services | Invalid record at line 6: Invalid product ID (line=6, field='product_id')

2026-10-27 20:02:54,724 | WARNING | ecommerce_importer.services | Low stock warning: Product ID=106 ('Winter Jacket') has only 2 items left (threshold=5)

2026-10-27 20:02:54,724 | WARNING | ecommerce_importer.services | Invalid record at line 8: Category 'kitchen' is not allowed (permitted: ['books', 'clothing', 'electronics', 'home']) (line=8, field='category')

2026-10-27 20:02:54,724 | WARNING | ecommerce_importer.services | Invalid record at line 9: Price must be non-negative (>= 0) (line=9, field='price')

2026-10-27 20:02:54,728 | INFO | ecommerce_importer.utils | E-commerce inventory pipeline finished successfully in 0.005455 s

2026-10-27 20:02:54,728 | INFO | ecommerce_importer | Execution completed: Total=9, valid=4, invalid=5, low_stock=2, exported=4

8.3. Вміст вихідного файлу output/products.json


```
[

{

  "product_id": 101,

  "name": "Wireless Mouse",

  "category": "electronics",

  "price": 25.99,

  "quantity": 15

},

{

  "product_id": 102,

  "name": "Python Cookbook",

  "category": "books",

  "price": 45.0,

  "quantity": 3

},

{

  "product_id": 106,

  "name": "Winter Jacket",
```

"category": "clothing",

"price": 89.9,

"quantity": 2

},

{

"product_id": 109,

"name": "T-Shirt",

"category": "clothing",

"price": 19.99,

"quantity": 50

}

]

9. Практичні експерименти з надійності

9.1. Експеримент 1. Порівняння Strict mode та Tolerant mode

При зміні конфігураційного параметра skip_invalid: false у файлі config.yaml програма переходить у режим **Strict mode**.

PS D:\Робочий стіл\проф. python\ecommerce_importer> python src/ecommerce_importer/main.py

2026-10-27 20:04:28,217 | INFO | ecommerce_importer.utils | E-commerce inventory pipeline started

2026-10-27 20:04:28,218 | WARNING | ecommerce_importer.services | Low stock warning: Product ID=102 ('Python Cookbook') has only 3 items left (threshold=5)

2026-10-27 20:04:28,219 | WARNING | ecommerce_importer.services | Invalid record at line 4: Product name cannot be empty (line=4, field='name')

2026-10-27 20:04:28,222 | ERROR | ecommerce_importer.utils | E-commerce inventory pipeline failed due to an error

Traceback (most recent call last):

```
  File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\utils.py", line 15, in logged_operation

    yield

  File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\main.py", line 32, in main

    exported_count = export_products_json(

        products,

        config.output_path,

    )

  File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\exporters.py", line 19, in export_products_json

    for product in products:

                   ^^^^^^^^

  File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\services.py", line 34, in import_products

    product = parse_product(

        row,

        line_number=line_number,

        allowed_categories=allowed_categories,

    )

  File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\validators.py", line 27, in parse_product

    raise RecordValidationError(

    ...<3 lines>...

    )
```

ecommerce_importer.exceptions.RecordValidationError: Product name cannot be empty (line=4, field='name')

2026-10-27 20:04:28,228 | ERROR | ecommerce_importer | Application error: Product name cannot be empty (line=4, field='name')

Порівняльна таблиця результатів:

Параметр DOCX	**Strict mode (skip_invalid: false) DOCX**	**Tolerant mode (skip_invalid: true) DOCX**
Оброблено записів	3 (зупинка на 4-му)	9 (усі записи файлу)
Valid	2	4
Invalid	1 (перший викликав зупинку)	5
Low stock warnings	1	2
Exported	0 (файл не пошкоджено, запис скасовано)	4
Завершення application	SystemExit(1) через RecordValidationError	Exit code 0 (успішно)

Висновки щодо режимів:

Tolerant mode доцільно застосовувати для фонової пакетної синхронізації каталогів чи імпорту товарів від зовнішніх постачальників, де помилка в окремому товарі не повинна зупиняти додавання решти асортименту.

Strict mode є критично необхідним для фінансових або складських транзакцій, де цілісність даних є абсолютним пріоритетом і наявність навіть одного битого рядка вимагає повної відміни завантаження.

9.2. Експеримент 2. Порівняння пам'яті: Streaming проти Eager Loading

Для оцінки ефективності використання пам'яті було проведено дослідження за допомогою модуля tracemalloc та скрипта experiment_memory.py на датасетах різного розміру:

PS D:\Робочий стіл\проф. python\ecommerce_importer> python src/ecommerce_importer/experiment_memory.py

--- Тестування для 10,000 записів ---

Генерація 10,000 рядків у data\benchmark_10000.csv...

Eager: Час = 0.1878 s | Пікова RAM = 4.15 MB

Streaming: Час = 0.0987 s | Пікова RAM = 0.04 MB

--- Тестування для 100,000 записів ---

Генерація 100,000 рядків у data\benchmark_100000.csv...

Eager: Час = 2.9861 s | Пікова RAM = 41.30 MB

Streaming: Час = 1.7155 s | Пікова RAM = 0.05 MB

--- Тестування для 500,000 записів ---

Генерація 500,000 рядків у data\benchmark_500000.csv...

Eager: Час = 8.7520 s | Пікова RAM = 207.40 MB

Streaming: Час = 8.2724 s | Пікова RAM = 0.05 MB

Кількість записів DOCX	**Eager time (s) DOCX**	**Streaming time (s) DOCX**	**Eager peak RAM (MB) DOCX**	**Streaming peak RAM (MB) DOCX**
10 000	0.1878	0.0987	4.15	0.04
100 000	2.9861	1.7155	41.30	0.05
500 000	8.7520	8.2724	207.40	0.05

Висновок: У жадібному варіанті (Eager) використання оперативної пам'яті масштабується лінійно $O(N)$ від обсягу датасету (сягаючи 207.40 MB на 500 тисячах рядків). Потоковий підхід (Streaming) підтримує споживання оперативної пам'яті на константному мінімальному рівні (0.04–0.05 MB, $O(1)$) незалежно від обсягу вхідного файлу, унеможливлюючи помилки вичерпання пам'яті (MemoryError).

9.3. Експеримент 3. Exception Chaining

Для дослідження механізму збереження початкової причини помилки було запущено скрипт experiment_chaining.py з передачею нечислового ідентифікатора товару "abc_wrong_id".

PS D:\Робочий стіл\проф. python\ecommerce_importer> python src/ecommerce_importer/experiment_chaining.py

=== Запуск Експерименту 3: Exception Chaining ===

1. Перехоплено доменний виняток: Invalid product ID (line=15, field='product_id')

2. Пряма першопричина (__cause__): ValueError("invalid literal for int() with base 10: 'abc_wrong_id'")

3. Повний ланцюжок винятків у Traceback:

Traceback (most recent call last):

  File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\validators.py", line 14, in parse_product

    product_id = int(row.get("product_id", "").strip())

ValueError: invalid literal for int() with base 10: 'abc_wrong_id'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):

  File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\experiment_chaining.py", line 23, in <module>

    parse_product(

    ~~~~~~~~~~~~~^

        bad_row,

        ^^^^^^^^

        line_number=15,
    ~~~~~~~~~~~~~

```
    ^^^^^^^^^^^^^^^^
    allowed_categories={"electronics"},
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  )
  ^
 File "D:\Робочий стіл\проф. python\ecommerce_importer\src\ecommerce_importer\validators.py", line 18, in parse_product
  raise RecordValidationError(
  ...<3 lines>...
  ) from error
ecommerce_importer.exceptions.RecordValidationError: Invalid product ID (line=15, field='product_id')
```

[УСПІХ] Ланцюжок ValueError -> RecordValidationError успішно зафіксовано!

Висновок: Використання конструкції raise ... from error записує початковий низькорівневий виняток ValueError в атрибут __cause__ доменного винятку RecordValidationError. Це дозволяє у звітах та логах одночасно бачити предметне пояснення дефекту з номером рядка та зберегти точну технічну інформацію про причину виникнення помилки для зневадження.

9.4. Експеримент 4. Атомарний вивід та обробка збоїв

Для підтвердження надійності атомарного експорту було виконано скрипт experiment_atomic.py, що імітує раптовий збій джерела даних (RuntimeError) під час запису другого елемента масиву.

```
PS D:\Робочий стіл\проф. python\ecommerce_importer> python src/ecommerce_importer/experiment_atomic.py
1. Створено початковий файл: products.json
  Вміст до аварії: [
{
```

"product_id": 101,

"name": "Initial Product",

"category": "electronics",

"price": 50.0,

"quantity": 10

}

]

2. Спроба експорту через генератор з помилкою...

3. Перехоплено очікувану помилку: Failed to export data to output\products.json

Причина винятку: RuntimeError('Раптовий аварійний збій у генераторі під час експорту!')

4. Перевірка цілісності файлової системи:

Чи існує тимчасовий файл 'products.json.tmp'?: False (має бути False)

Чи зберігся попередній основний файл 'products.json'?: True (має бути True)

Вміст основного файлу після аварії:

[

{

"product_id": 101,

"name": "Initial Product",

"category": "electronics",

"price": 50.0,

"quantity": 10

}

]

Висновок: Експеримент підтвердив, що у разі аварійного збою посеред експорту:

1. Тимчасовий файл .tmp негайно видаляється (False), запобігаючи засміченню файлової системи.
2. Основний робочий файл залишається повністю неушкодженим (True), не містить обірваних рядків або невалідного JSON-синтаксису.
3. Виняток коректно загортається в доменний клас DataExportError.

Висновок:

У ході лабораторної роботи №5 реалізовано надійний модуль імпорту, валідації та експорту товарів інтернет-магазину (Варіант №3).

Створено доменну ієрархію винятків із контекстом рядків і полів, а механізм exception chaining (raise ... from) підтвердив збереження першопричини ValueError в __cause__. Завантаження конфігурації YAML забезпечило дотримання принципу fail-fast ще на старті. Потоковий ввід (Streaming I/O) на базі генераторів довів стале споживання оперативної пам'яті $O(1)$: лише 0.05 MB RAM проти 207.40 MB при жадібному зчитуванні для 500 000 рядків. Атомарний експорт у JSON через тимчасовий .tmp файл гарантував збереження цілісності результату при збоях. Порівняно режими Strict та Tolerant, налаштовано логування залишків (low-stock), автоматизовано таймінг власним контекстним менеджером, а всі зміни зафіксовано в Git.